

Dokumen Operasional Deployment

Dokumen ini merangkum proses *deployment* aplikasi ke lingkungan *cluster* Google Kubernetes Engine (GKE), baik melalui metode manual maupun otomatis dengan CI/CD.

1. Deployment Manual

Deployment manual dilakukan untuk kebutuhan *testing* atau *deployment* awal. Proses ini melibatkan serangkaian perintah baris yang dieksekusi secara berurutan.

Langkah-langkah Deployment

Berikut adalah tahapan manual untuk men-deploy aplikasi ke GKE:

Langkah 0: Inisiasi Cluster GKE dengan Terraform

Tahap ini hanya dilakukan sekali untuk membuat atau memperbarui infrastruktur *cluster*.

1. Inisiasi Terraform

```
# Lakukan di folder `tf/`  
terraform init
```

2. Buat Rencana Eksekusi

```
terraform plan -out tfplan
```

3. Terapkan Konfigurasi

```
terraform apply -auto-approve tfplan
```

4. Konfigurasi **kubectl**

```
# Lihat konteks saat ini  
kubectl config current-context --project primeval-rune-467212-t9  
  
# Ubah konteks kubectl agar berinteraksi dengan cluster GKE  
gcloud container clusters get-credentials wondr-desktop-cluster --zone asia-southeast1-a --project primeval-rune-467212-t9
```

Langkah 1: Build dan Push Docker Image

Setelah infrastruktur siap, *image* Docker untuk *frontend* dan *backend* dibangun dan di-*push* ke Google Container Registry (GCR).

Catatan Penting: Pastikan file Firebase JSON sudah tersedia di direktori *frontend* dan *backend* sebelum proses *build*.

a. Frontend

- **Build Image:** Gunakan `--build-arg` untuk menyisipkan *environment variable* yang tidak bisa diatur melalui `.yaml` Kubernetes.

```
# Lakukan di root folder aplikasi
docker build \
  --build-arg VITE_BACKEND_BASE_URL=GANTISAYA \
  --build-arg VITE_VERIFICATOR_BASE_URL=[https://verificator-secure-
onboarding-system-441501015598.asia-southeast1.run.app](https://verificator-
secure-onboarding-system-441501015598.asia-southeast1.run.app) \
  --build-arg VITE_FIREBASE_API_KEY=AIzaSyCTXgqBktnmUo8z5VkxMuwBpLkBGZ_syj0
\
  ...
-t asia.gcr.io/primeval-rune-467212-t9/wondr-desktop-fe:latest \
./frontend-secure-onboarding-system
```

- **Push Image:**

```
docker push asia.gcr.io/primeval-rune-467212-t9/wondr-desktop-fe:latest
```

b. Backend

- **Build Image:**

```
docker build -t asia.gcr.io/primeval-rune-467212-t9/wondr-desktop-be:latest
./backend-secure-onboarding-system
```

- **Push Image:**

```
docker push asia.gcr.io/primeval-rune-467212-t9/wondr-desktop-be:latest
```

Langkah 2: Apply Manifest Kubernetes

Manifest Kubernetes (`.yaml` files) diterapkan untuk membuat *deployment*, *service*, dan konfigurasi lainnya di *cluster* GKE.

- **Database:**

```
kubectl apply -f ./k8s/db/db-deployment.yaml
kubectl apply -f ./k8s/db/db-secrets.yaml
kubectl apply -f ./k8s/db/db-service.yaml
```

- **Backend:**

```
kubectl apply -f ./k8s/be/be-secrets.yaml
kubectl apply -f ./k8s/be/be-healthcheck.yaml
kubectl apply -f ./k8s/be/be-configmaps.yaml
kubectl apply -f ./k8s/be/be-deployment.yaml
kubectl apply -f ./k8s/be/be-service.yaml
```

- **Frontend:**

```
kubectl apply -f ./k8s/fe/fe-secrets.yaml
kubectl apply -f ./k8s/fe/fe-deployment.yaml
kubectl apply -f ./k8s/fe/fe-service.yaml
```

- **Network (Ingress):**

```
kubectl delete -f ./k8s/network/ingress.yaml
kubectl apply -f ./k8s/network/ingress.yaml
```

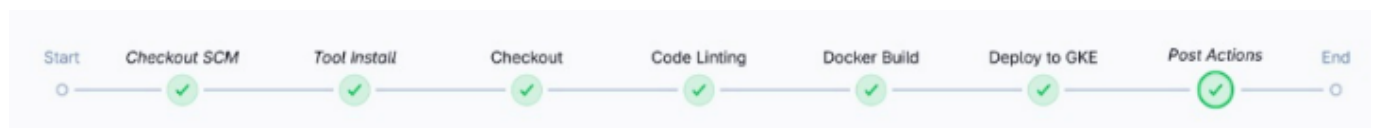
- **Verifikasi Deployment:**

```
kubectl describe pods [nama_pods]
```

2. Deployment Otomatis dengan CI/CD

Pipeline CI/CD diimplementasikan menggunakan Jenkins untuk mengotomatisasi seluruh alur kerja, dari *checkout* kode hingga *deployment* di lingkungan produksi. Berbeda dari pendekatan monolitik, pipeline ini dipisahkan menjadi dua *job* yang independen untuk *frontend* dan *backend*.

2.1 Pipeline Frontend



Pipeline ini fokus pada proses *build*, *test*, dan *deployment* aplikasi *frontend*.

Tahapan dalam Pipeline:

1. **Checkout:** Mengambil kode sumber dari repositori `frontend-secure-onboarding-system`.
2. **Code Linting:** Menjalankan `npm install` dan `npm run lint` untuk memeriksa kualitas kode.
3. **Docker Build & Push:**
 - Mengambil kunci rahasia (`VITE_FIREBASE_API_KEY`) dari Google Cloud Secret Manager.
 - Membangun *image* Docker dengan *environment variable* yang dimasukkan melalui `--build-arg`.
 - Memberi tag *image* dengan nomor *build* Jenkins (`${env.BUILD_NUMBER}`).
 - Mendorong *image* ke Google Container Registry (GCR).
4. **Deploy to GKE:**
 - Mengganti *image tag* di file `k8s/deployment.yaml` dengan nomor *build* saat ini.
 - Mengkonfigurasi `kubectl` untuk terhubung ke *cluster* GKE.
 - Menerapkan konfigurasi `deployment.yaml` ke *cluster*.

Environment Variables Kunci:

- `VITE_BACKEND_BASE_URL`: URL *backend* yang diakses oleh *frontend*.
- `VITE_VERIFICATOR_BASE_URL`: URL layanan verifikator.
- `VITE_FIREBASE_*`: Variabel konfigurasi untuk layanan Firebase Auth.

2.2 Pipeline Backend



Pipeline ini fokus pada proses *build*, *test*, dan *deployment* aplikasi *backend*.

Tahapan dalam Pipeline:

1. **Checkout:** Mengambil kode sumber dari repositori `backend-secure-onboarding-system`.
2. **Unit Test:** Menjalankan `mvn package` untuk mengeksekusi semua *unit test* dan menghasilkan laporan. Laporan ini kemudian diarsipkan dengan `junit 'target/surefire-reports/*.xml'`.
3. **Sonarqube Test:**
 - Mengambil token SonarQube dari Google Cloud Secret Manager.
 - Menjalankan analisis kode dengan Maven (`mvn clean verify sonar:sonar`).
4. **Docker Build & Push:**
 - Mengambil kredensial Firebase Service Account (`firebase-otp-cred`) dari Google Cloud Secret Manager dan menyimpannya sebagai file.
 - Memindahkan file kredensial ke direktori sumber (`src/main/resources`).
 - Membangun *image* Docker dengan `build-arg` untuk kredensial tersebut.
 - Mendorong *image* ke Google Container Registry (GCR) dengan tag nomor *build* saat ini.
5. **Deploy to GKE:**
 - Mengganti *image tag* di file `k8s/deployment.yaml` dengan nomor *build* saat ini.
 - Mengkonfigurasi `kubectl` untuk terhubung ke *cluster* GKE.
 - Menerapkan konfigurasi `deployment.yaml` ke *cluster*.

Environment Variables Kunci:

- **SONAR_PROJECT_KEY**: Kunci proyek untuk SonarQube.
- **SONAR_URL**: URL server SonarQube.
- **IMAGE_TAG**: Nomor *build* Jenkins yang digunakan sebagai tag *image*.

2.3 Konfigurasi Jenkins

- **Lingkungan Jenkins**: Menggunakan Docker-in-Docker (*dind*) untuk menjalankan Jenkins di dalam container, yang memiliki akses ke Docker socket *host*.
 - **Tools**: Menggunakan plugin penting seperti Maven, JDK, dan NodeJS.
 - **Secrets Management**: Mengelola kredensial sensitif (GCR Service Account Key, Firebase Private Key, SonarQube Token) menggunakan **Google Cloud Secret Manager** dan diakses langsung dari dalam pipeline.
-

3. Troubleshooting Umum

- **Ruang Disk Penuh**: Jika proses *build* Jenkins terhenti, periksa penggunaan disk VM. Solusinya adalah meningkatkan ukuran disk VM di Google Cloud.
- **Izin Akses**: Jika terjadi *permission denied*, pastikan user Jenkins memiliki izin yang memadai (misalnya, dengan menggunakan **chmod**) pada direktori yang relevan.
- **IP Dinamis**: Pastikan alamat IP eksternal VM Jenkins diubah menjadi statis untuk menghindari masalah konektivitas.

4. Catatan

- **Urutan Ingress**: Dalam konfigurasi *ingress*, urutan *path* sangat penting. *Path* yang paling spesifik, seperti **/api/auth**, harus diletakkan di atas *path* yang lebih umum, seperti **/**, untuk memastikan *routing* berjalan dengan benar.
- **Build History**: Dokumentasikan setiap riwayat *build* di Jenkins (sukses atau gagal) dengan deskripsi yang jelas untuk mempermudah proses *debugging* dan audit.